Stacks

COMP 182 · Lecture 08

Last in, first out

Implementations, expressions, backtracking, and recursion

Java example: StackDemo.java

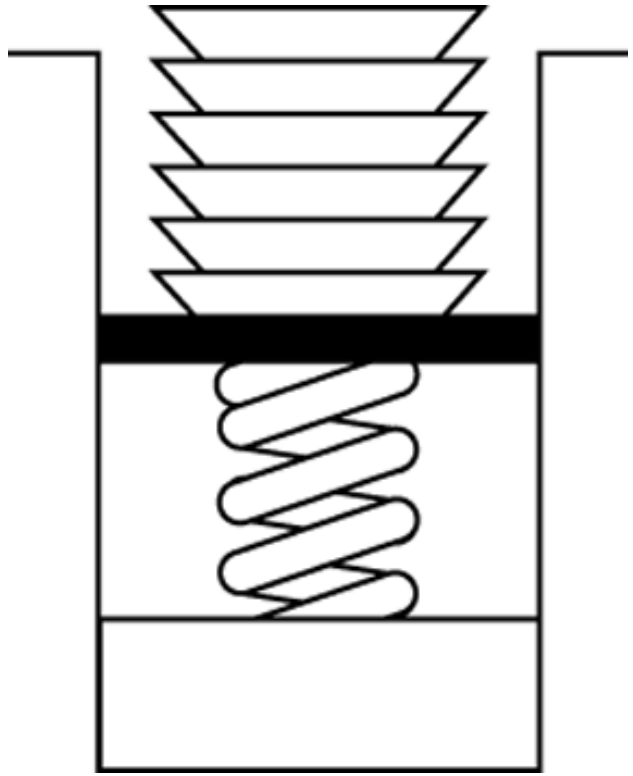
Last in, first out

A stack gives access to one end, called the top.

push adds an item at the top. pop removes the top item.

The most recently pushed item leaves first.

Push 10, 20, 30. Successive pops return 30, 20, 10.



Java example: StackDemo.java

The stack ADT

Operation	Effect
Create	Start with an empty stack
isEmpty()	Report whether the stack has no items
push(item)	Add item at the top
pop()	Remove and return the top item
peek()	Return the top item without removing it
clear()	Remove every item

Specify the empty-stack behavior for pop and peek as part of the interface.

Java example: GenericStack.java

Design the ADT around the problem

Application	What the stack remembers	Why LIFO helps
Maze or graph search	The current route and unfinished choices	Return to the most recent choice
Display backward	Items read in their original order	Remove items in reverse order

Choose the operations the solution needs.

Keep storage details behind the stack interface.

Java example: StackDemo.java

Checking balanced braces

```
stack = empty stack
for each character ch:
  if ch == '{': stack.push(ch)
  else if ch == '}':
    if stack.isEmpty(): return false
    stack.pop()
return stack.isEmpty()
```

Balanced:
abc{defg{ijk}{l{mn}}op}qr

Unbalanced:
abc{def}}{ghij{kl}m

Each closing brace must match an earlier opening brace.

At the end, no unmatched opening brace may remain.

Java example: BracketChecker.java

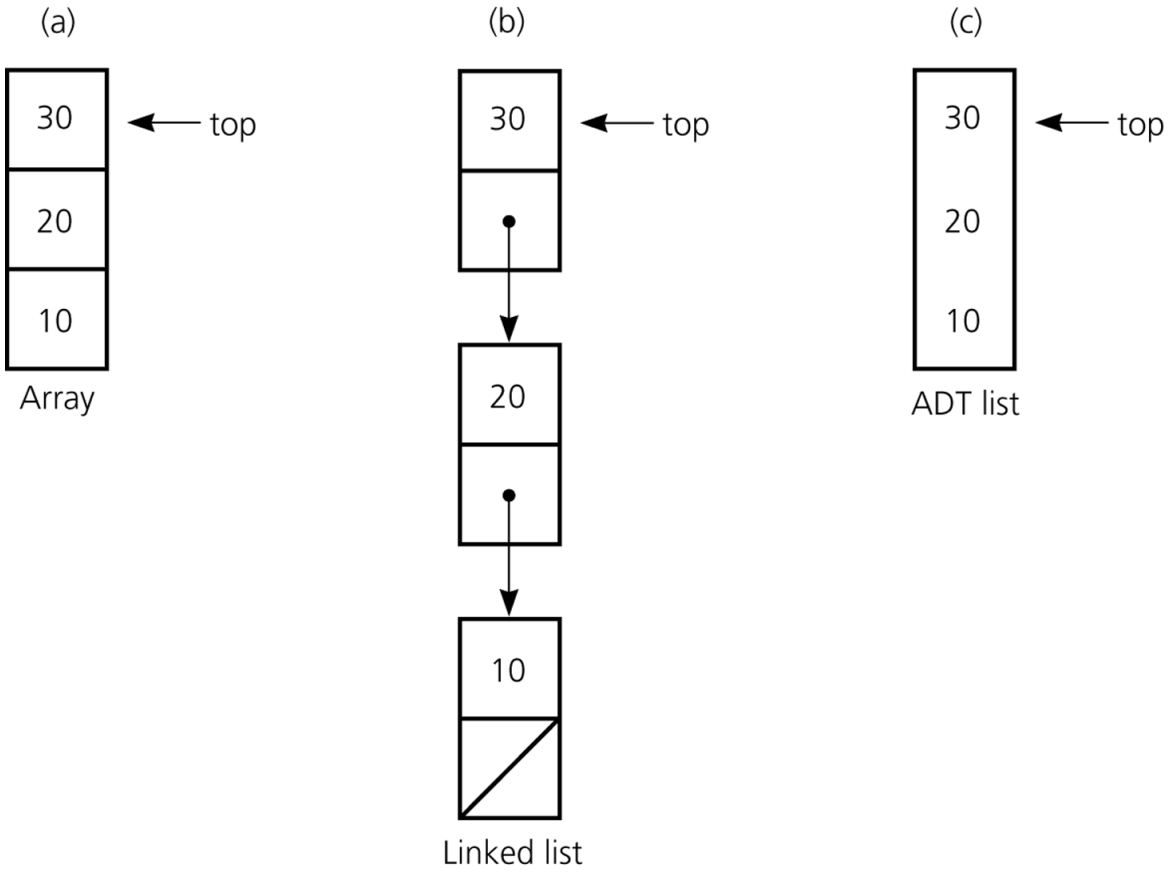
Array and linked implementations

Implementation	Where is the top?	push	pop / peek
Fixed array	Last occupied position	O(1), if capacity remains	O(1)
Resizable array	Last occupied position	Amortized O(1)	O(1)*
Singly linked list	Head node	O(1)	O(1)

* Assumes no shrinking copy during that pop. A growth operation can make one push O(n).

Java example: LinkedStack.java

The same stack, different storage



All three views have 30 at the top, then 20, then 10.

An array tracks the top index.

A linked implementation tracks the head reference.

A list ADT can supply the storage operations.

Java example: LinkedStack.java

Choosing stack storage

```
// ArrayList-backed stack, top at the end
void push(E item) { list.add(item); }

E pop() {
  if (list.isEmpty())
    throw new NoSuchElementException();
  return list.remove(list.size() - 1);
}
```

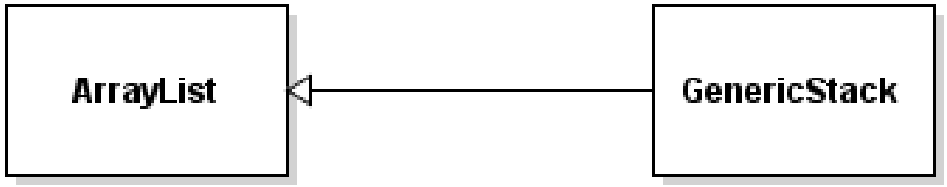
ArrayList push is amortized O(1). Linked-head push is O(1).

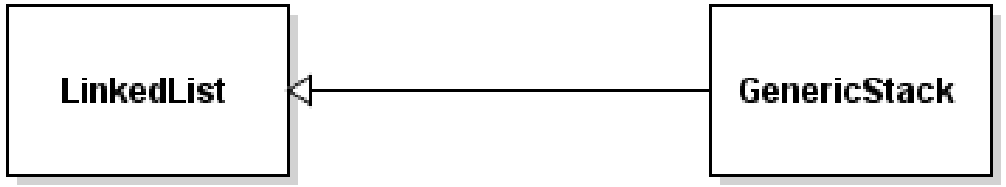
Allocation, memory locality, and workload affect real performance.

Java example: GenericStack.java

Composition and inheritance

Inheritance exposes the list's full public API





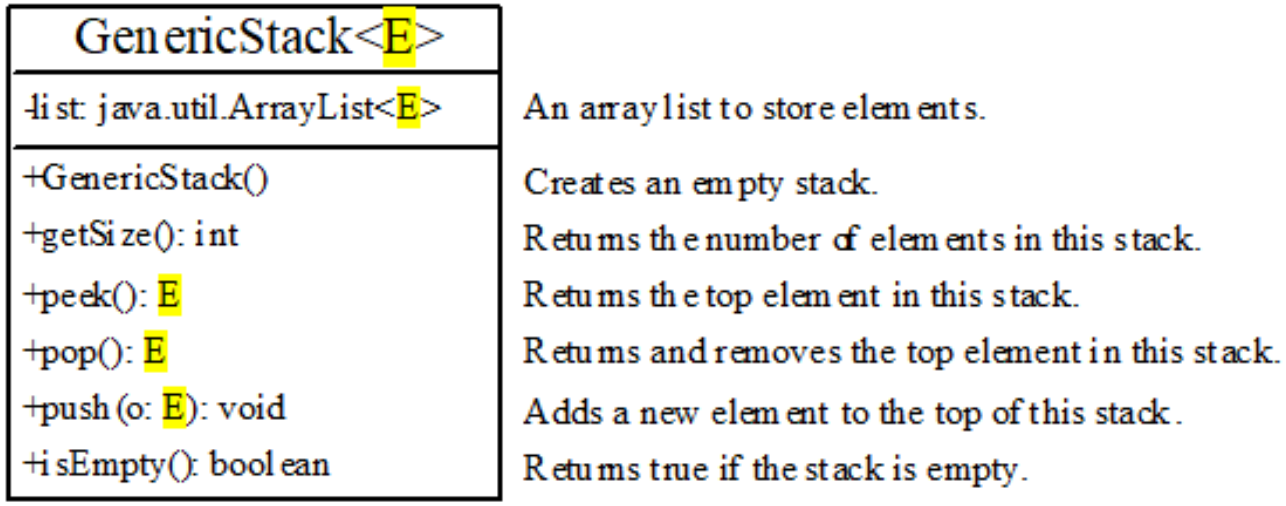
Composition keeps a private list behind a stack interface





Java example: GenericStack.java

GenericStack<E>: the public interface



getSize() reports the number of items.

push, pop, and peek operate at the top.

isEmpty() checks whether getSize() is zero.

A queue uses FIFO order and needs a different interface.

Java example: GenericStack.java

Java stack APIs

```
import java.util.ArrayDeque;
import java.util.Deque;

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
int top = stack.peek();     // 20
int removed = stack.pop(); // 20
```

java.util.Stack<E> extends Vector and remains a legacy API.

For new code, Java recommends Deque and its implementations.

Java example: StackDemo.java

Infix and postfix expressions

Notation	Example	Meaning
Infix	2 * (3 + 4)	Operators appear between operands
Postfix	2 3 4 + *	Each operator follows its operands

Convert infix to postfix using a stack of operators.

Evaluate postfix using a stack of operand values.

Java example: ExpressionDemo.java

Converting infix to postfix

```
operators = empty stack; output = empty list
for each token:
  operand: send it to output
  "(": push it onto operators
  ")": pop to output until "("; discard "("
  operator op:
    while operators not empty and top != "(":
      if prec(top) < prec(op): break
      pop top to output
    push op
after input: pop remaining operators to output
```

For +, -, *, /
with left associativity

2 * (3 + 4)

Postfix:
2 3 4 + *

Java example: ExpressionDemo.java

Evaluating postfix: 2 3 4 + *

Token	Action	Stack, bottom to top
2	Push 2	[2]
3	Push 3	[2, 3]
4	Push 4	[2, 3, 4]
+	Pop 4 and 3. Push 3 + 4.	[2, 7]
*	Pop 7 and 2. Push 2 * 7.	[14]

Pop the right operand first, then the left. Compute left op right.

Java example: ExpressionDemo.java

Flight connections as a search problem

HPAir asks whether any route connects an origin to a destination.

A route may require one or more connecting flights.

Search must follow the direction of each flight.

A stack can hold the current route while exploring alternatives.

Goal: find a route or report that none exists.

Java example: FlightSearchDemo.java

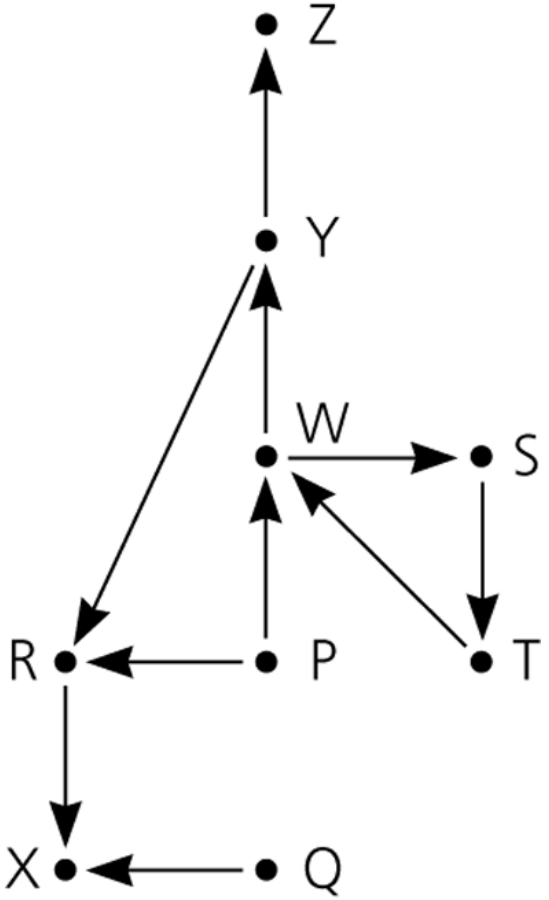
A directed flight graph

Each city is a vertex. Each direct flight is a directed edge.

An outgoing neighbor is a city reached by one direct flight.

A directed path follows the arrow on every edge.

The map contains a cycle: W, S, T, W.



Java example: FlightSearchDemo.java

Depth-first search: push and backtrack

Action, searching P to Z	Current route, bottom to top
Start at P	[P]
Explore R, then X	[P, R, X]
Backtrack from X, then R	[P]
Explore W, then S, then T	[P, W, S, T]
Skip visited W. Backtrack from T and S.	[P, W]
Explore Y, skip visited R, then reach Z.	[P, W, Y, Z]

Push when advancing. Pop when a city has no unvisited outgoing neighbor.

Java example: FlightSearchDemo.java

Representing flights with adjacency lists

City	Outgoing neighbors, in visit order
P	R, W
Q	X
R	X
S	T
T	W
W	S, Y
X	None
Y	R, Z
Z	None

Neighbor order determines the search trace. It does not change reachability.

Java example: FlightSearchDemo.java

Iterative depth-first search

```
visited = {origin}
stack.push(frame(origin, iterator(adj[origin])))
while not stack.isEmpty():
  f = stack.peek()
  if f.city == destination:
    return cities(stack, bottom-to-top)
  if not f.neighbors.hasNext(): stack.pop()
  else:
    next = f.neighbors.next()
    if next not in visited:
      visited.add(next)
      stack.push(frame(next, iterator(adj[next])))
return "no route"
```

Each frame stores a city and its next unexplored neighbor.

Return immediately when the destination is found.

Java example: FlightSearchDemo.java

Recursive search: the stopping rules

At the destination: return true.

At an unvisited neighbor: recursively search from that city.

At a visited neighbor: skip it to avoid repeated work and cycles.

After every outgoing choice fails: return false.

Mark each city before exploring its outgoing edges.

Java example: FlightSearchDemo.java

Recursive depth-first search

```
search(city, destination, visited):
  visited.add(city)
  if city == destination: return true
  for next in adj[city]:
    if next not in visited:
      if search(next, destination, visited):
        return true
  return false
```

Start with an empty visited set.

With adjacency lists: O(V + E) time and O(V) extra search space.

Java example: FlightSearchDemo.java

Recursion uses a stack of calls

Execution point	Active calls, bottom to top
Call factorial(3)	[factorial(3)]
Reach factorial(1)	[factorial(3), factorial(2), factorial(1)]
Return 1 from factorial(1)	[factorial(3), factorial(2)]
Return 2 from factorial(2)	[factorial(3)]
Return 6 from factorial(3)	Empty

Each call stores local state and a return point. The latest unfinished call returns first.

Java example: RecursionDemo.java